Project Executable Files

Date	03 October 2026
Team ID	2
Project Name	FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks	3 Marks

Project Executable Files

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes — schema defined in app/database.py, created automatically on start; ER diagram in README
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	NA — local deployment as per project brief
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA — web application
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes — tests/ (245 pytest tests), Locust report and CPU/memory samples in reports/
10	Demo video or walkthrough (if required)	To be added by the team (link in repository README)

Step 2: File / Folder Structure

```
FitBuddy/
├── app/
│   ├── main.py                    FastAPI app, static files, router, table creation
│   ├── config.py                  .env settings (API key, models, admin password)
│   ├── routes.py                  HTML + JSON routes
│   ├── database.py                SQLAlchemy models (users, plans) + CRUD helpers
│   ├── schemas.py                 Pydantic models (UserInput, FeedbackRequest, WorkoutPlan)
│   ├── gemini_client.py           shared Gemini client, structured JSON, retry, fallback
│   ├── gemini_generator.py        generate_workout_gemini()
│   ├── gemini_flash_generator.py  generate_nutrition_tip_with_flash()
│   ├── updated_plan.py            update_workout_plan()
│   ├── demo_data.py               demo-mode plans, tips, revisions
│   ├── templates/                 base, index, result, _plan, all_users, admin_login, error
│   └── static/                    css/style.css, images/gym-bg.jpg
├── tests/                         test_routes.py, test_security.py, test_ai_hardening.py,
│                                  test_demo_safety.py, conftest.py, locustfile.py,
resource_monitor.py
├── reports/                       Locust report, perf summary, CPU/memory samples, repo
audit, logs
├── screenshots/                   page screenshots
├── .env.example  .gitignore  .gitattributes  pytest.ini  requirements.txt  README.md
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Local: http://127.0.0.1:8000 (API docs: /docs)
Login Credentials (Demo)	Admin dashboard (/view-all-users): password = value of ADMIN_PASSWORD in your .env (default in .env.example: `admin123`). End users need no login.
Platform / Hosting Provider	Local machine with Uvicorn (Windows, macOS or Linux)
Repository Link	https://github.com/Chethana45/fitbuddy
Demo Video Link	To be added by the team (link in repository README)

Step 4: Run Instructions

```
1. Install Python 3.11+ and clone the repository:
   git clone https://github.com/Chethana45/fitbuddy.git
2.                                                          t):
   cd FitBuddy
3. Create and activate a virtual environment:
   python -m venv venv
   venv\Scripts\activate            (Mac/Linux: source venv/bin/activate)
4. Install dependencies:
   pip install -r requirements.txt
5. (Optional, for real AI) copy .env.example to .env and set GEMINI_API_KEY
   (free key from https://aistudio.google.com/apikey). Without a key the app
   runs in demo mode.
6. Start the server:
   uvicorn app.main:app --reload
7. Open http://127.0.0.1:8000, generate a plan, submit feedback, and visit
   /view-all-users (admin) and /docs (API).
8. Run tests:  pytest
```

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Google's gemini-3.8-flash sometimes returns HTTP 503 (high demand), adding 13–20 s	Automatic retry and fallback to gemini-3.5-flash-lite; or set the light model as primary in .env
2	Free-tier Gemini API has rate limits	Demo mode for testing; load test run in demo mode
3	SQLite and a single admin password suit local / small use	SQLAlchemy allows switching to PostgreSQL; add per-user accounts in a future phase